Unidade I
Estruturas de dados para representação de grafos

Prof. Henrique Batista da Silva

Adaptado do material prof.ª Anna Tostes (PUC Minas)
Implementações foram adaptadas de [Ziviani, 2006]

Representação de um Grafo

- Dado um grafo $G = (V, E)$:
  - V = conjunto de vértices.
  - E = conjunto de arestas, que pode ser representado pelo subconjunto de $V \times V$.
- O tamanho da entrada de dados é medido em termos do:
  - Número de vértices $|V|$.
  - Número de arestas $|E|$.
- Se G é conexo, então $|E| \geq |V| - 1$.

Representação de um Grafo

- Convenção I (Notação):
  - Dentro e somente dentro da notação assintótica os símbolos V e E significam respectivamente $|V|$ e $|E|$.
  - Se um algoritmo "executa em tempo $\mathrm{O}(V + E)$" é equivalente a dizer que "executa em tempo $\mathrm{O}(|V| + |E|)$".

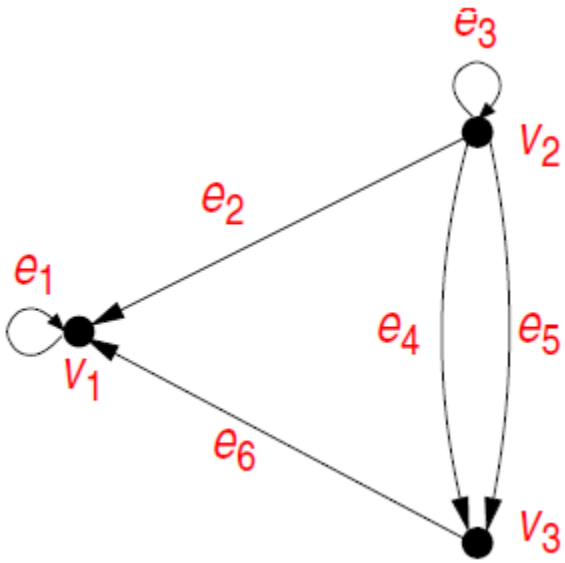
- Convenção II (Em pseudocódigo):
  - O conjunto V de vértices de G é representado por $V[G]$.
  - O conjunto E de arestas de G é representado por $E[G]$.
  - Os conjuntos V e E são vistos como atributos de G.

Representação de um Grafo: Estruturas de Dados

- Matriz de adjacência:
  - Forma preferida de representar grafos densos ($|E| \approx |V|^2$).
  - Indica rapidamente ($\mathrm{O}(1)$) se existe uma aresta conectando dois vértices.
- Lista de adjacência:
  - Representação normalmente preferida.
  - Provê uma forma compacta de representar grafos esparsos ($|E| \approx |\mathrm{V}|$).
- Matriz de incidência:
  - Representação que inclui vértice e aresta.
- As duas primeiras formas acima são as principais formas de representar um grafo.

Matriz de adjacência e grafo dirigido

Seja o grafo dirigido abaixo:



Este grafo pode ser representado por uma matriz $A = (a_{ij})$, onde (a_{ij}) representa o número de arestas de v_i para v_j.

Matriz de Adjacência:

$$A = \begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \end{array} \begin{array}{c} \begin{array}{ccc} v_1 & v_2 & v_3 \end{array} \\ \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 2 \\ 1 & 0 & 0 \end{bmatrix} \end{array}$$

Matriz de adjacência e grafo dirigido

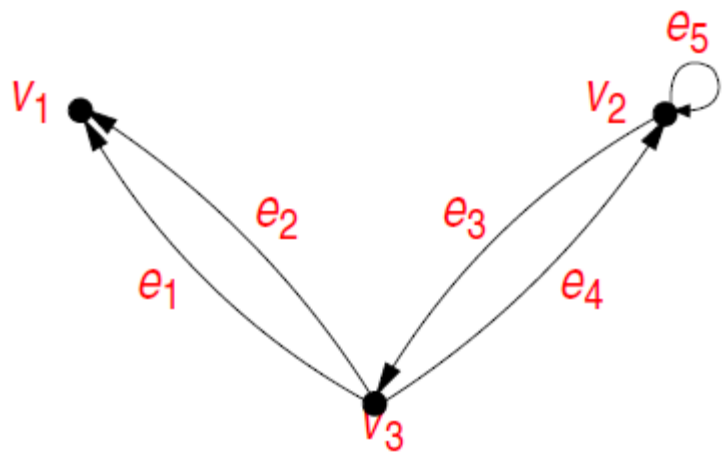
Definição: Seja G um grafo dirigido com vértices $v_1, v_2, \dots, v_n$. A matriz de adjacência de G é a matriz $A = (a_{ij})(A[1 \dots n, 1 \dots n])$, definida como:

$$a_{ij} = \text{\# de arestas de } v_i \text{ para } v_j, \forall i, j = 1, 2, \dots n.$$

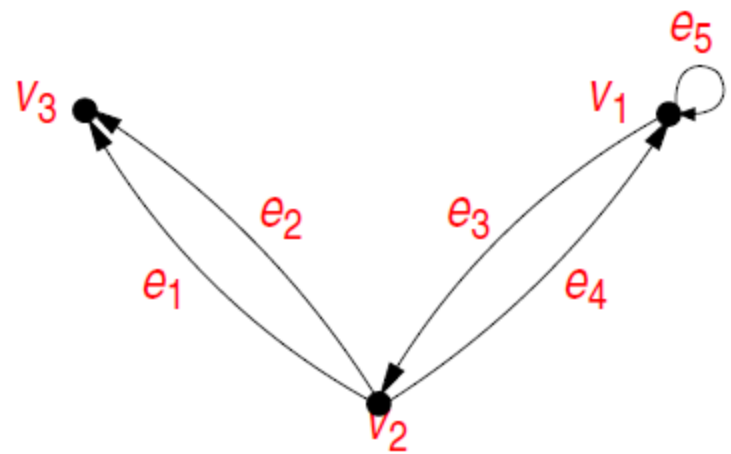
- Valor diferente de zero na diagonal principal: laço.
- Valor igual a 1 na entrada (a_{ij}): uma única aresta de v_i para v_j.
- Valores maiores que 1 na entrada (a_{ij}): arestas paralelas de v_i a v_j.
- Espaço: $O(V^2)$.

Matriz de adjacência e grafo dirigido

- Exemplos de matriz de adjacência:



$$A = \begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \end{array} \begin{array}{c} \begin{array}{ccc} v_1 & v_2 & v_3 \end{array} \\ \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 1 \\ 2 & 1 & 0 \end{bmatrix} \end{array}$$



$$A = \begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \end{array} \begin{array}{c} \begin{array}{ccc} v_1 & v_2 & v_3 \end{array} \\ \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 2 \\ 0 & 0 & 0 \end{bmatrix} \end{array}$$

Matriz de adjacência e grafo dirigido

Dada a matriz de adjacência de um grafo:

$$A = \begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \\ v_4 \end{array} \begin{array}{c} \begin{array}{cccc} v_1 & v_2 & v_3 & v_4 \end{array} \\ \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 2 \\ 0 & 0 & 1 & 1 \\ 2 & 1 & 0 & 0 \end{bmatrix} \end{array}$$

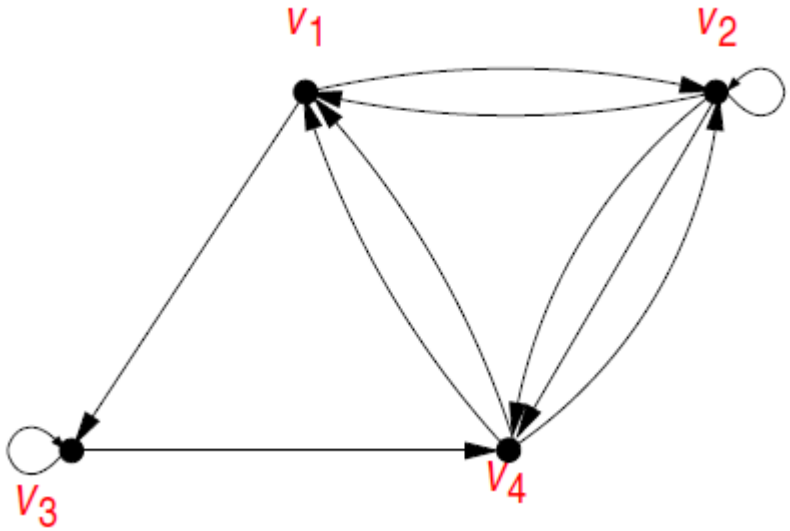
Agora desenhe o grafo

Matriz de adjacência e grafo dirigido

Dada a matriz de adjacência de um grafo:

$$A = \begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \\ v_4 \end{array} \begin{array}{c} \begin{array}{cccc} v_1 & v_2 & v_3 & v_4 \end{array} \\ \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 2 \\ 0 & 0 & 1 & 1 \\ 2 & 1 & 0 & 0 \end{bmatrix} \end{array}$$

Um possível desenho deste grafo



Matriz de adjacência e grafo **não** dirigido

Definição: Seja G um grafo não dirigido com vértices $v_1, v_2, \dots,$ v_n. A matriz de adjacência de G é a matriz $A = (a_{ij})$ sobre o conjunto de inteiros não negativos tal que:

$$a_{ij} = \text{\# de arestas conectando } v_i \text{ a } v_j, \forall i, j = 1, 2, \dots n.$$

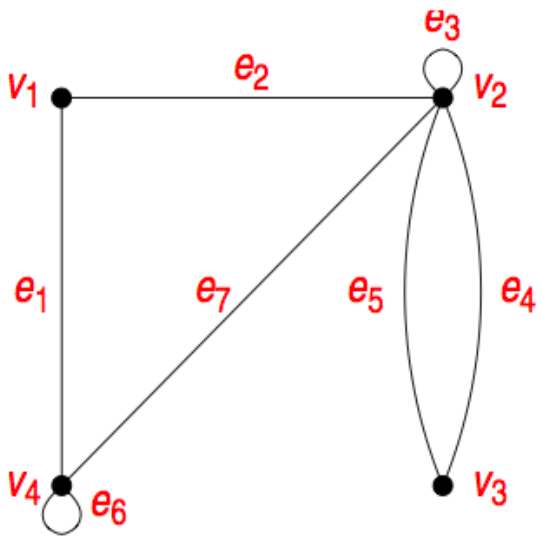
- Valor diferente de zero na diagonal principal: laço.
- Valor igual a 1 na entrada (a_{ij}): uma única aresta de v_i a v_j.
- Valores maiores que 1 na entrada (a_{ij}): arestas paralelas de v_i a v_j.
- Espaço: $O(V^2)$.

Matriz de adjacência e grafo **não** dirigido

- Definição: Seja G um grafo não dirigido com vértices $v_1, v_2, \dots, v_n$. A matriz de adjacência de G é a matriz $A = (a_{ij})$ sobre o conjunto de inteiros não negativos tal que:

$$a_{ij} = \# \text{ de arestas conectando } v_i \text{ a } v_j, \forall i, j = 1, 2, \dots n.$$

Seja o grafo abaixo:



Matriz de adjacência correspondente:

$$A = \begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \\ v_4 \end{array} \begin{array}{c} \begin{array}{cccc} v_1 & v_2 & v_3 & v_4 \end{array} \\ \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 1 & 2 & 1 \\ 0 & 2 & 0 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix} \end{array}$$

Matriz de adjacência: análise

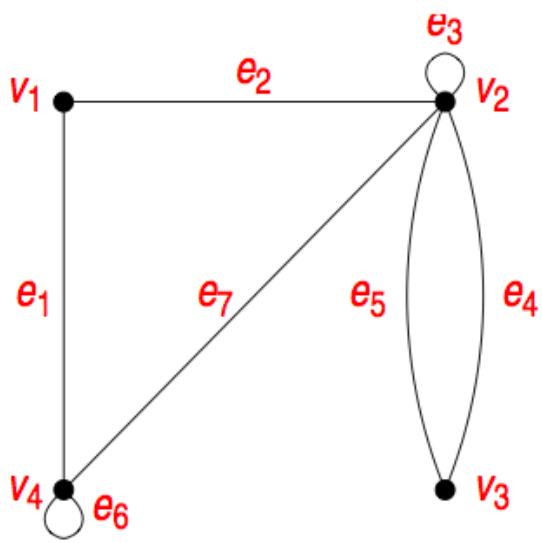
- Deve ser utilizada para grafos densos, onde $|E|$ é próximo de $|V|^2$. ($|E| \approx |V|^2$).
- O tempo necessário para acessar um elemento é independente de $|V|$ ou $|E|$.
- É muito útil para algoritmos em que necessitamos saber com rapidez se existe uma aresta ligando dois vértices.
- A maior desvantagem é que a matriz necessita $O(V^2)$ de espaço.
- Ler ou examinar a matriz tem complexidade de tempo $O(V^2)$.

Matriz de incidência

- Definição: Seja G um grafo não dirigido com vértices $v_1, v_2, \dots, v_n$ e arestas $e_1, e_2, \dots, e_m$. A matriz de incidência de G é a matriz $M = (m_{ij})$ de tamanho $n \times m$ sobre o conjunto de inteiros não negativos tal que:

$$m_{ij} = \begin{cases} 1 & \text{quando a aresta } e_j \text{ é incidente a } v_i \\ 0 & \text{caso contrário} \end{cases}$$

Seja o grafo:



Matriz de incidência correspondente:

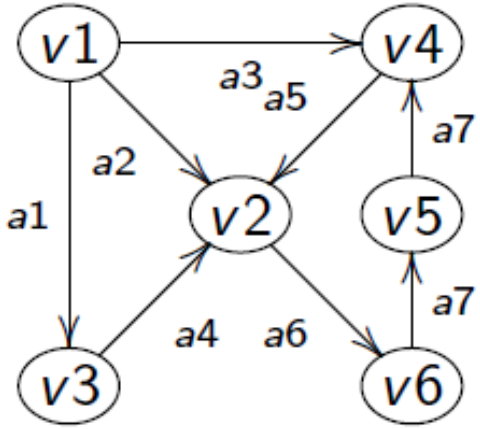
$$M = \begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \\ v_4 \end{array} \begin{array}{c} \begin{array}{ccccccc} e_1 & e_2 & e_3 & e_4 & e_5 & e_6 & e_7 \end{array} \\ \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 & 1 \end{bmatrix} \end{array}$$

Matriz de incidência

- Definição: Seja G um grafo não dirigido com vértices $v_1, v_2, \dots, v_n$ e arestas $e_1, e_2, \dots, e_m$. A matriz de incidência de G é a matriz $M = (m_{ij})$ de tamanho $n \times m$ sobre o conjunto de inteiros não negativos tal que:

$$m_{ij} = \begin{cases} 1 & \text{quando a aresta } e_j \text{ é incidente a } v_i \\ 0 & \text{caso contrário} \end{cases}$$

Seja o grafo:



Matriz de incidência correspondente:

	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8
v_1	1	1	1	0	0	0	0	0
v_2	0	-1	0	-1	-1	1	0	0
v_3	-1	0	0	1	0	0	0	0
v_4	0	0	-1	0	1	0	-1	0
v_5	0	0	0	0	0	0	1	-1
v_6	0	0	0	0	0	-1	0	1

Matriz de adjacência

```
public class GrafoMatriz
  {
    public class Aresta
    {
      private int v1, v2, peso;
      public Aresta(int v1, int v2, int peso)
      {
        this.v1 = v1; this.v2 = v2; this.peso = peso;
      }
      public int get_peso() { return this.peso; }
      public int get_v1() { return this.v1; }
      public int get_v2() { return this.v2; }
    }

    private int [,]mat;
    private int numVertices;
    private int []pos;
```

Matriz de adjacência

```
public GrafoMatriz (int numVertices)
    {
        this.mat = new int[numVertices, numVertices];
        this.pos = new int[numVertices];
        this.numVertices = numVertices;

        for (int i = 0; i < this.numVertices; i++)
        {
            for (int j = 0; j < this.numVertices; j++)      this.mat[i,j] = 0;

            this.pos[i] = -1;
        }
    }

    public void insereAresta(int v1, int v2, int peso)    {     this.mat[v1, v2] = peso;        }
    public bool existeAresta(int v1, int v2, int peso)        {           return (this.mat[v1, v2] > 0);       }

    public bool listaAdjVazia(int v)
    {
        for (int i = 0; i < this.numVertices; i++)
            if (this.mat[v, i] > 0)
                return false;
            return true;
    }
```

Matriz de adjacência

```
public Aresta primeiroListaAdj(int v)
    {
      // Retorna a primeira aresta que o v\'ertice v participa ou
      // se a lista de adjac\^encia de v for vazia
      this.pos[v] = -1; return this.proxAdj(v);
    }
    public Aresta proxAdj(int v)
    {
      //Retorna a pr\'oxima aresta que o v\'ertice v participa ou}@
      //se a lista de adjac\^encia de v estiver no fim
      this.pos[v]++;
      while ((this.pos[v] < this.numVertices) && (this.mat[v, this.pos[v]] == 0))
        this.pos[v]++;

      if (this.pos[v] == this.numVertices)
        return null;

      else return new Aresta(v, this.pos[v], this.mat[v, this.pos[v]]);
    }
```

Matriz de adjacência

```
public Aresta retiraAresta(int v1, int v2)    {
        if (this.mat[v1, v2] == 0)      return null;
        else
        {
           Aresta aresta = new Aresta(v1, v2, this.mat[v1, v2]);
           this.mat[v1, v2] = 0;
           return aresta;
        }
     }

     public void imprime ()     {
        Console.WriteLine("   ");
        for (int i = 0; i < this.numVertices; i++)     Console.WriteLine (i + "   ");

        Console.WriteLine ();

        for (int i = 0; i < this.numVertices; i++)      {
           Console.WriteLine (i + "  ");
           for (int j = 0; j < this.numVertices; j++)   Console.WriteLine(this.mat[i, j] + "   ");

           Console.WriteLine ();
        }
     }
```

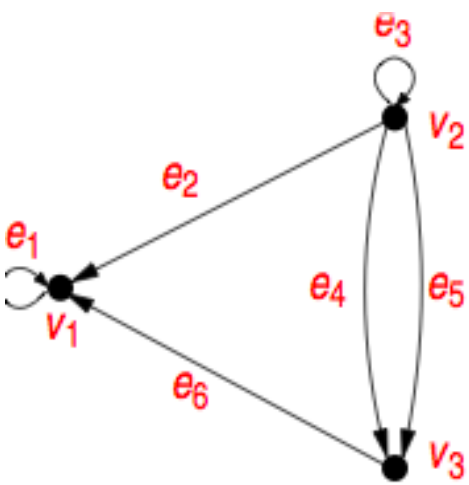
Matriz de adjacência

```
public int get_numVertices() { return this.numVertices; }

    public GrafoMatriz grafoTransposto()
    {
        GrafoMatriz grafoT = new GrafoMatriz (this.numVertices);
        for (int v = 0; v < this.numVertices; v++)
            if (!this.listaAdjVazia(v))
            {
                Aresta adj = this.primeiroListaAdj(v);
                while (adj != null)
                {
                    grafoT.insereAresta(adj.get_v2(), adj.get_v1(), adj.get_peso());
                    adj = this.proxAdj(v);
                }
            }

        return grafoT;
    }

}
```

Lista de adjacência

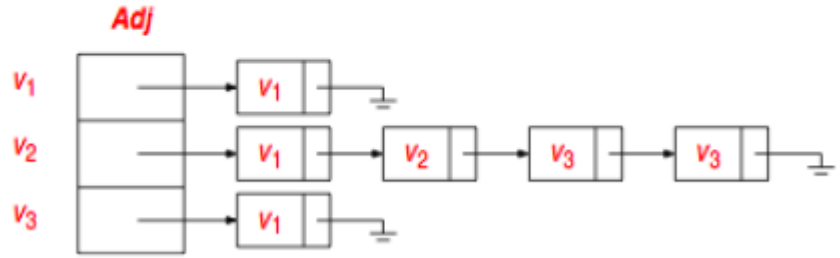
- Em grafo dirigido:

Seja o grafo dirigido abaixo:

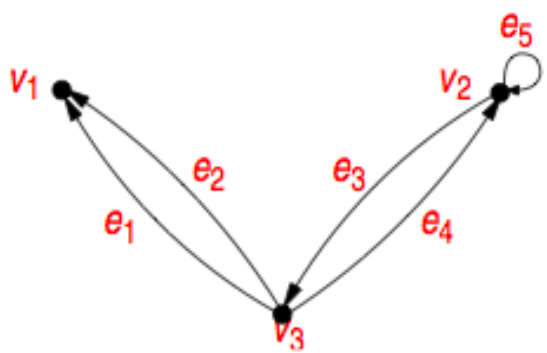


Este grafo pode ser representado por uma lista de adjacência Adj:

$$Adj[v_1] = [v_1]$$
$$Adj[v_2] = [v_1, v_2, v_3, v_3]$$
$$Adj[v_3] = [v_1]$$

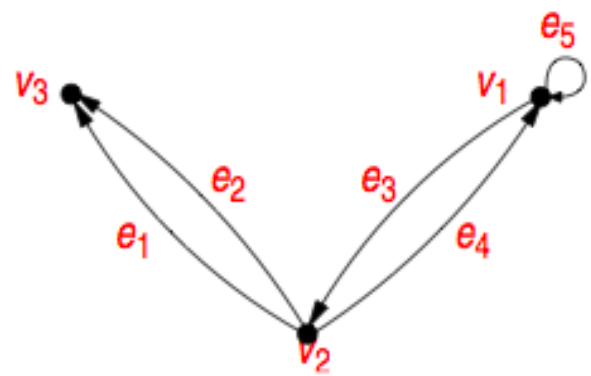


Lista de adjacência



Este grafo pode ser representado por uma lista de adjacência:

$$Adj[v_1] = []$$
$$Adj[v_2] = [v_2, v_3]$$
$$Adj[v_3] = [v_1, v_1, v_2]$$



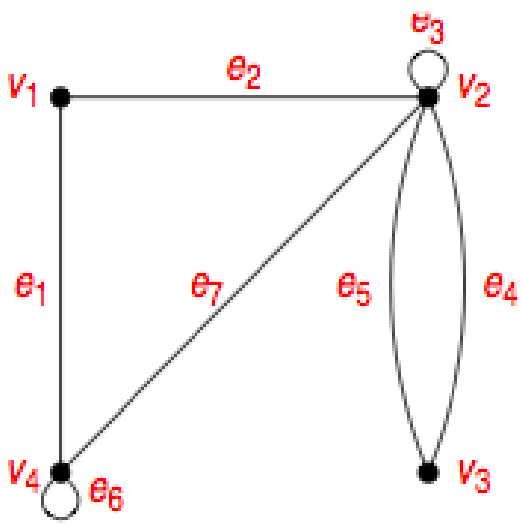
Este grafo pode ser representado por uma lista de adjacência:

$$Adj[v_1] = [v_1, v_2]$$
$$Adj[v_2] = [v_1, v_3, v_3]$$
$$Adj[v_3] = []$$

Lista de adjacência

- Em grafo não dirigido:

Dado o grafo:



Este grafo pode ser representado por uma lista de adjacência Adj:

$$Adj[v_1] = [v_2, v_4]$$
$$Adj[v_2] = [v_1, v_2, v_3, v_3, v_4]$$
$$Adj[v_3] = [v_2, v_2]$$
$$Adj[v_4] = [v_1, v_2, v_4]$$

Lista de adjacência

- Vetor Adj de $|V|$ listas, uma para cada vértice de V.
- Para cada vértice v a lista de adjacência de Adj$[v]$ contém os apontadores para todos os vértices adjacente a v no grafo.
  - Para grafos dirigidos e não dirigidos
- Tamanho de uma lista de adjacência:
  - Grafo dirigido: $|E|$, cada aresta aparece uma única vez.
  - Grafo não dirigido: $2|E|$, cada aresta aparece duas vezes na lista (entrada de v e entrada de u para (v, u)).
  - Espaço: $O(V^2)$.

Lista de adjacência

```
public class Grafo
  {
    public class Aresta
    {
      private int v1, v2, peso;

      public Aresta(int v1, int v2, int peso)
      {
        this.v1 = v1;
        this.v2 = v2;
        this.peso = peso;
      }
      public int get_peso() { return this.peso; }
      public int get_v1() { return this.v1; }
      public int get_v2() { return this.v2; }
    }

    private ListaRef [] adj;
    private int numVertices;
```

Lista de adjacência

```
public Grafo(int numVertices)
    {
      this.adj = new ListaRef[numVertices];
      this.numVertices = numVertices;

      for (int i = 0; i < this.numVertices; i++)
        this.adj[i] = new ListaRef();
    }

    public void insereAresta(int v1, int v2, int peso)
    {
      Celula item = new Celula(v2, peso);
      this.adj[v1].insere(item);
    }

    public bool existeAresta(int v1, int v2, int peso)
    {
      Celula item = new Celula(v2, peso);
      return (this.adj[v1].pesquisa(item) != null);
    }

    public bool listaAdjVazia(int v)    {    return this.adj[v].vazia();    }
```

Lista de adjacência

```
public Aresta primeiroListaAdj ( int v )
    {
      // Retorna a primeira aresta que o vértice v participa ou
      // null se a lista de adjacência de v for vazia
      Celula item = (Celula)this.adj[v].Primeiro();
      return item != null ? new Aresta(v, item.get_vertice(), item.get_peso()) : null;
    }

    public Aresta proxAdj ( int v )
    {
      // Retorna a próxima aresta que o vértice v participa ou
      // null se a lista de adjacência de v estiver no fim
      Celula item = (Celula)this.adj[v].proximo();
      return item != null ? new Aresta(v, item.get_vertice(), item.get_peso()) : null;
    }

    public Aresta retiraAresta ( int v1, int v2, int peso)
    {
      Celula chave = new Celula (v2, peso);
      Celula item = (Celula)this.adj[v1].retira(chave);
      return item != null ? new Aresta (v1, v2, item.get_peso()) : null;
    }
```

Lista de adjacência

```
public void imprime ( )
    {
      for (int i = 0; i < this.numVertices; i ++)
      {
        Console.WriteLine("Vertice " + i + " : " );
        Celula item = (Celula)this.adj[i].Primeiro();
        while ( item != null )
        {
          Console.WriteLine(" " + item.get_vertice() + " ( " + item.get_peso() + " ) ");
          item = (Celula)this.adj[i].proximo();
        }
      }
    }

    public int get_numVertices()
    {
      return this.numVertices;
    }

    public int get_grauSaidaVertice(int i)
    {
      return this.adj[i].quantidade();
    }
}
```

Lista de adjacência

```
public class Celula
  {
    int vertice, peso;
    public Celula(int v, int p) { this.vertice = v; this.peso = p; }

    public bool equals(Object obj)
    {
      Celula item = (Celula)obj;
      if ((this.vertice == item.vertice) && (this.peso == item.peso))
        return true;

      return false;
    }

    public int get_vertice() { return vertice; }
    public int get_peso() { return peso; }
  }
```

Lista de adjacência

- Vejam no código anterior que a classe Grafo tem um objeto privado “private ListaRef [] adj”, que é uma lista (um vetor) do tipo “ListaRef”. Esta é um Tipo de Abstrato de Dado (não é tipo da linguagem). Ou seja, o vetor “adj” é instanciado com a quantidade de vértices do grafo (informado pelo programa principal). Cada posição do vetor “adj” contém uma lista encadeada (auto referenciadas) com, inicialmente, zero elementos.
- A medida em que as arestas são inseridas no grafo, elementos são criados nas listas de seus respectivos vértices.

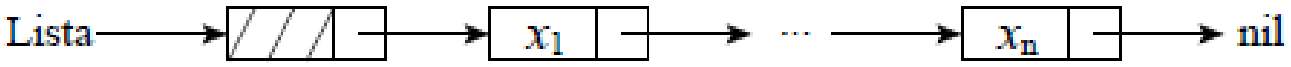
Implementação de Listas por meio de Estruturas Auto Referenciadas (extra)

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Também podem ser chamadas de listas encadeadas ou listas ligadas.
- Cada item da lista contém a informação que é necessária para alcançar o próximo item.
- Permite utilizar posições não contíguas de memória.
- Inserir e retirar elementos:
  - Não há necessidade de deslocar os itens seguintes da lista.

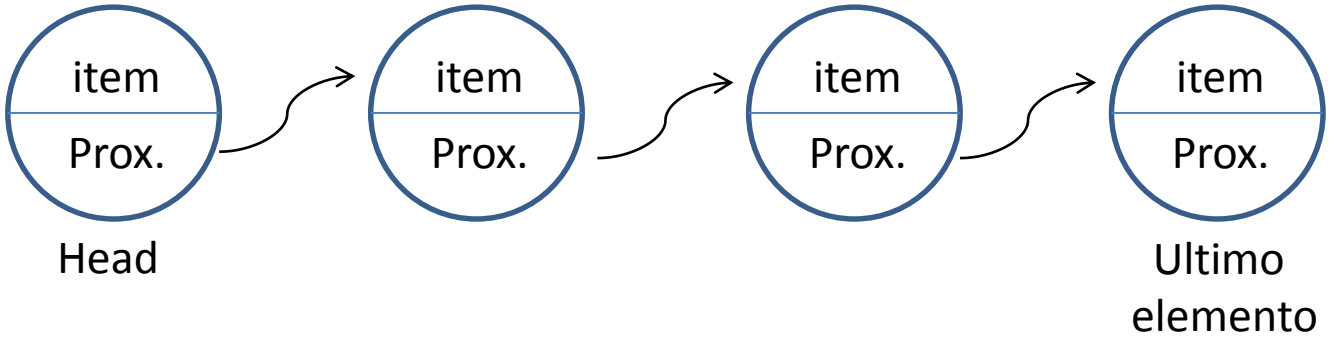
Implementação de Listas por meio de Estruturas Auto Referenciadas

- Há uma célula cabeça (head):
  - simplifica as operações sobre a lista



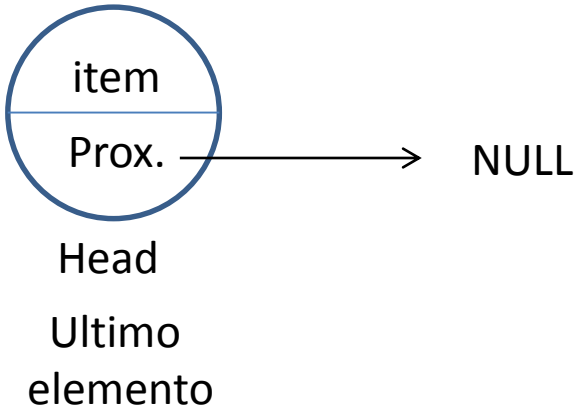
Implementação de Listas por meio de Estruturas Auto Referenciadas

- A lista é constituída de células:
  - Cada uma contém um item da lista e uma referência para a célula seguinte.



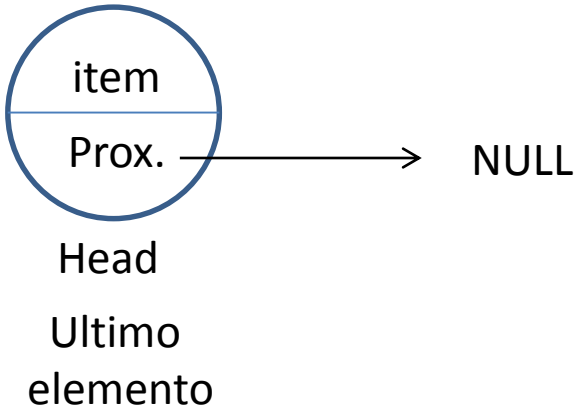
Implementação de Listas por meio de Estruturas Auto Referenciadas

- Construção de uma Lista vazia:



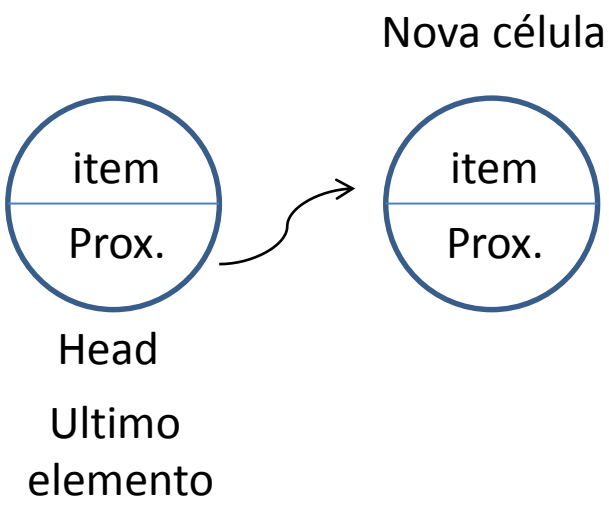
Implementação de Listas por meio de Estruturas Auto Referenciadas

- Inserção de um novo elemento na lista (aresta):



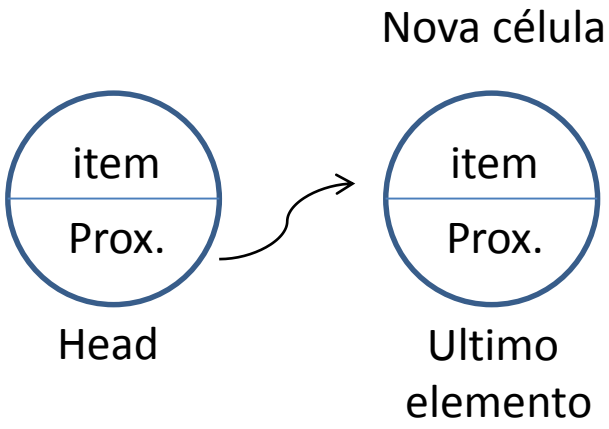
Implementação de Listas por meio de Estruturas Auto Referenciadas

- Inserção de um novo elemento na lista (aresta):



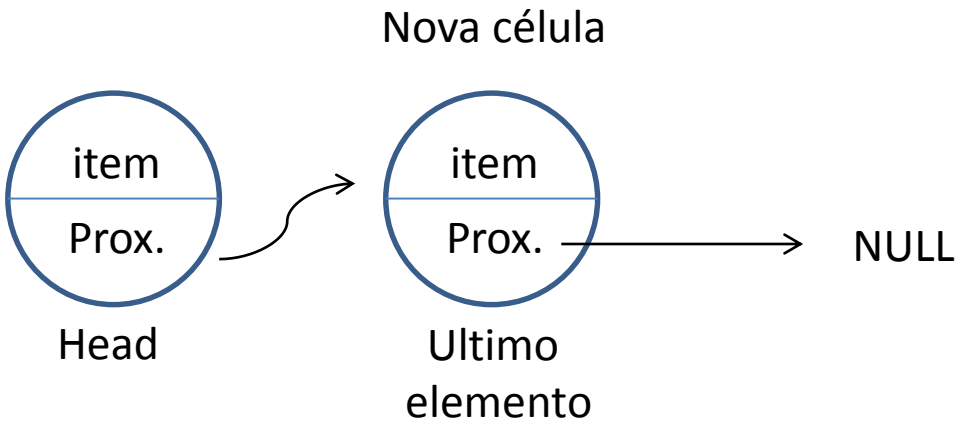
Implementação de Listas por meio de Estruturas Auto Referenciadas

- Inserção de um novo elemento na lista (aresta):



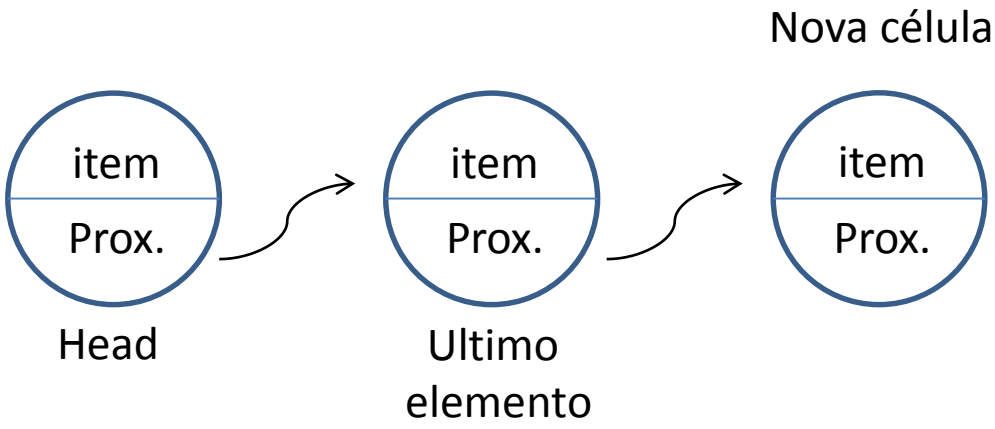
Implementação de Listas por meio de Estruturas Auto Referenciadas

- Inserção de um novo elemento na lista (aresta):



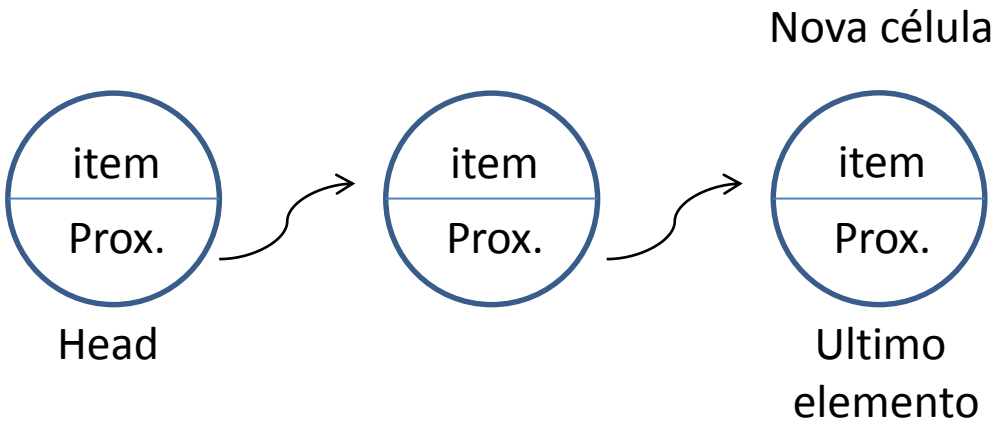
Implementação de Listas por meio de Estruturas Auto Referenciadas

- Inserção de um novo elemento na lista (aresta):



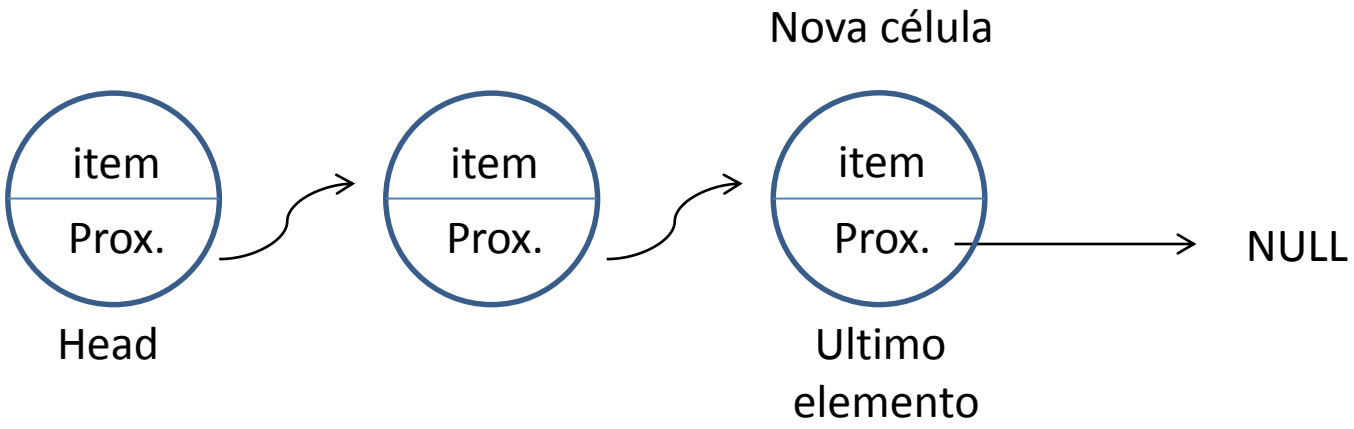
Implementação de Listas por meio de Estruturas Auto Referenciadas

- Inserção de um novo elemento na lista (aresta):



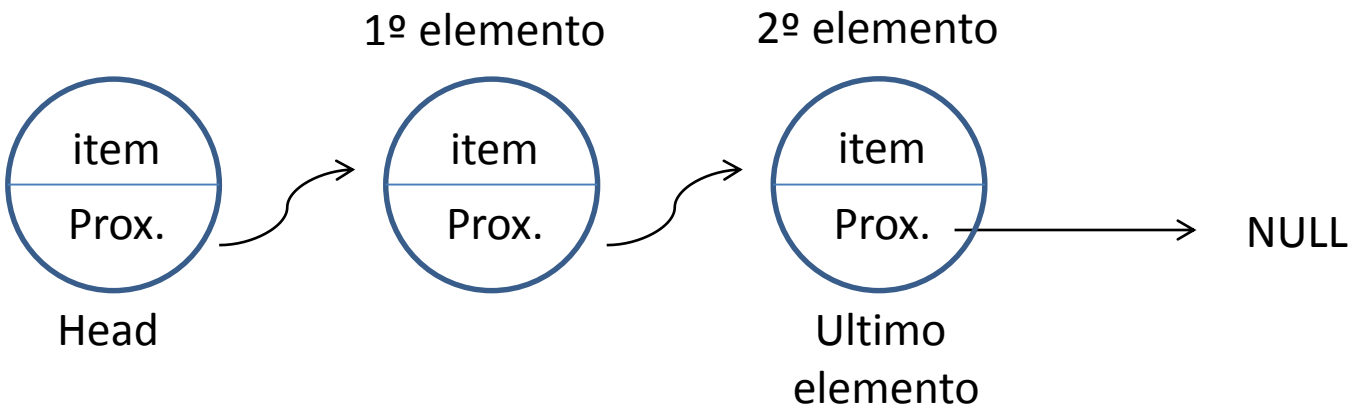
Implementação de Listas por meio de Estruturas Auto Referenciadas

- Inserção de um novo elemento na lista (aresta):



Implementação de Listas por meio de Estruturas Auto Referenciadas

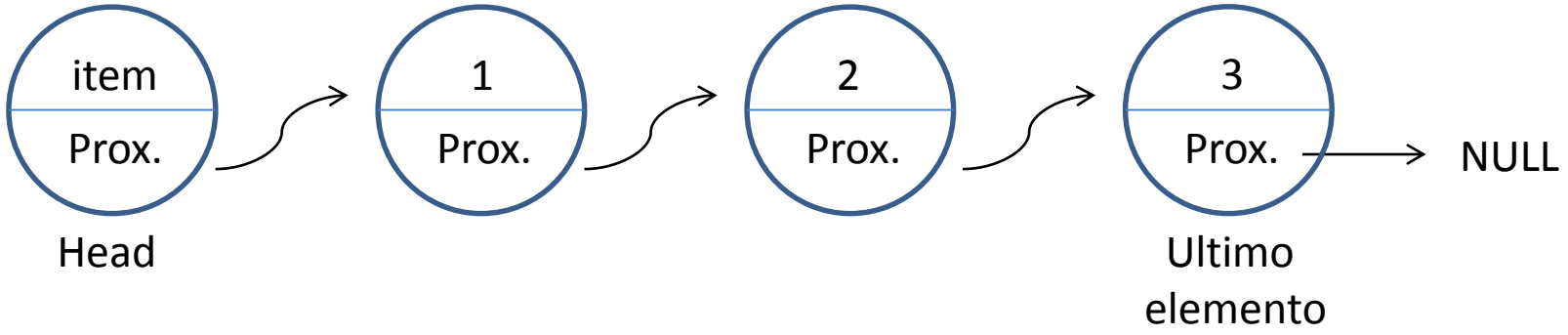
- Inserção de um novo elemento na lista (aresta):



Implementação de Listas por meio de Estruturas Auto Referenciadas

- Pesquisa de um elemento na lista:

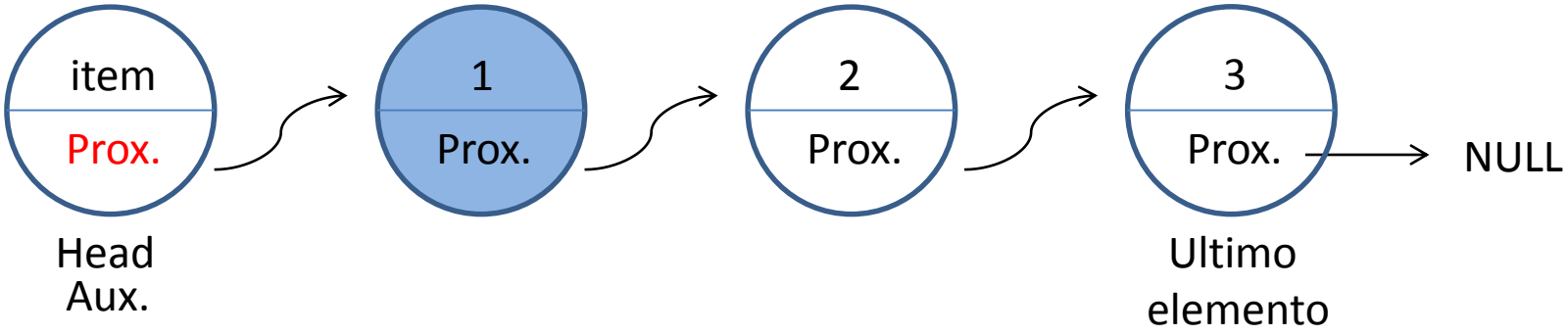
pesquisa(2);



Implementação de Listas por meio de Estruturas Auto Referenciadas

- Pesquisa de um elemento na lista:

pesquisa(2);

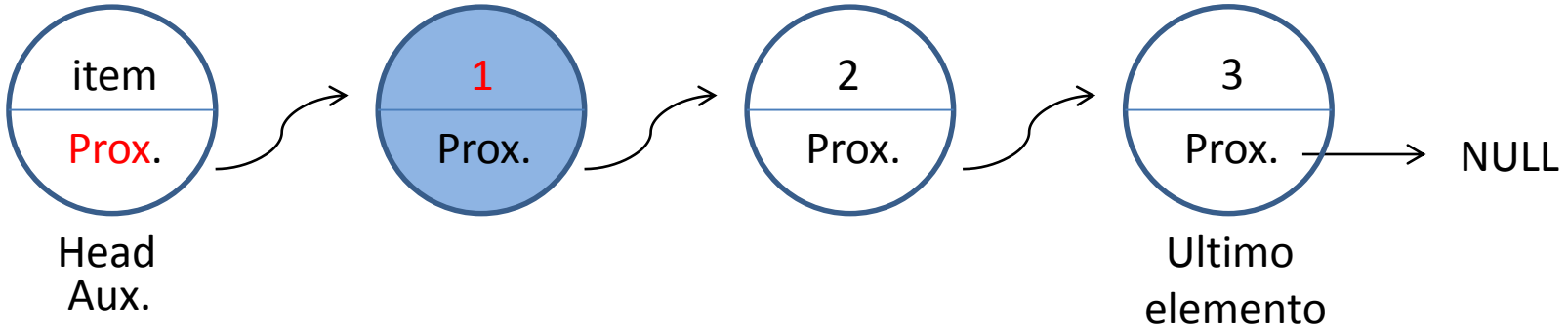



```
while (aux.prox != null ) {
    if (aux.prox.item == elemento)
        return aux.prox.item;
    aux = aux.prox;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Pesquisa de um elemento na lista:

pesquisa(2);

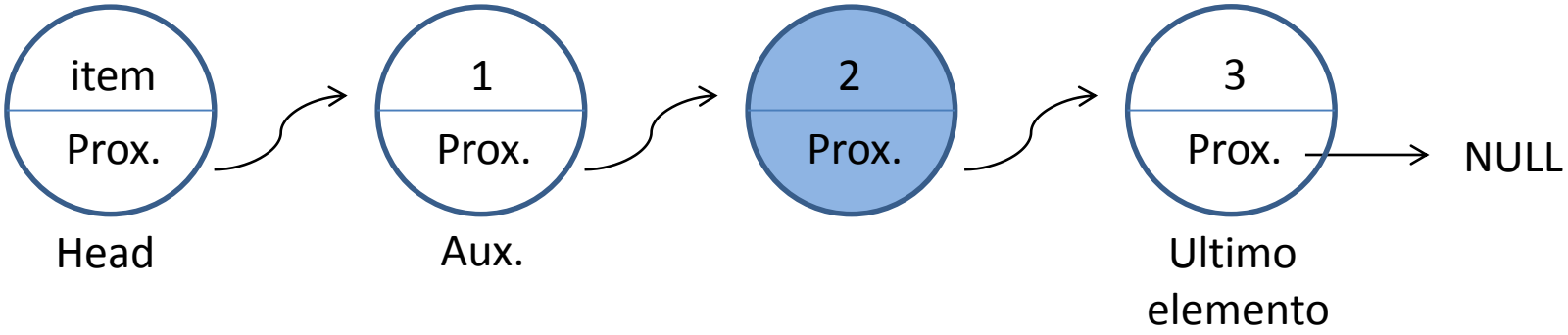



```
while (aux.prox != null ) {
    if (aux.prox.item == elemento)
        return aux.prox.item;
    aux = aux.prox;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Pesquisa de um elemento na lista:

pesquisa(2);

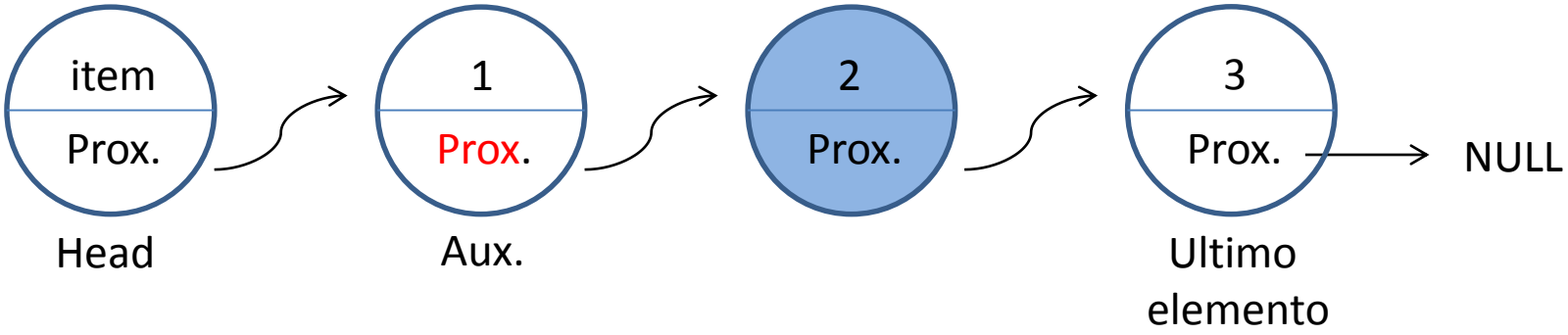



```
while (aux.prox != null ) {
    if (aux.prox.item == elemento)
        return aux.prox.item;
    aux = aux.prox;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Pesquisa de um elemento na lista:

pesquisa(2);

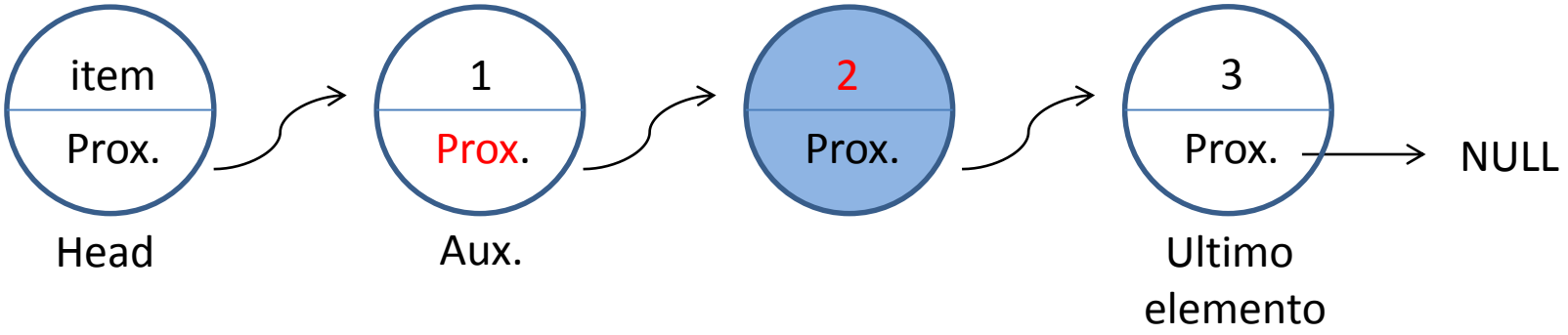



```
while (aux.prox != null ) {
    if (aux.prox.item == elemento)
        return aux.prox.item;
    aux = aux.prox;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Pesquisa de um elemento na lista:

pesquisa(2);

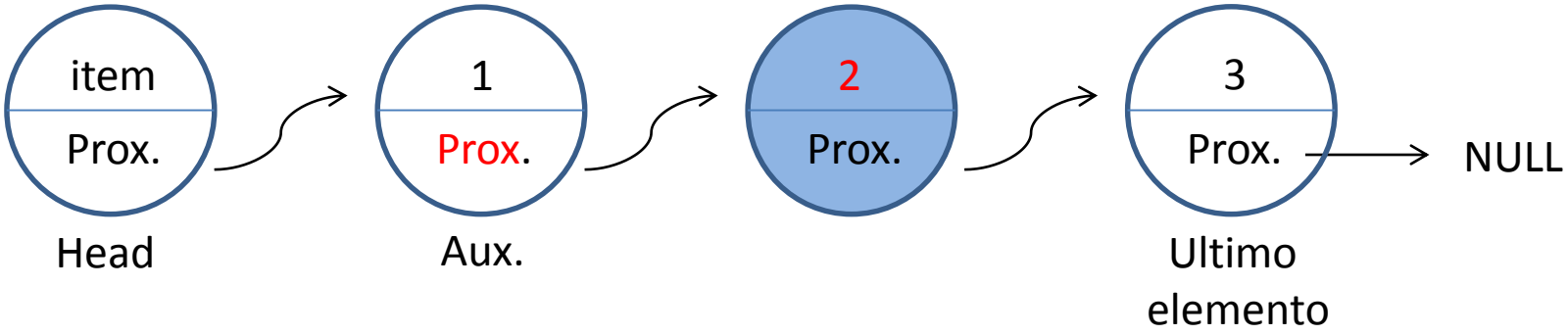



```
while (aux.prox != null ) {
    if (aux.prox.item == elemento)
        return aux.prox.item;
    aux = aux.prox;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Pesquisa de um elemento na lista:

pesquisa(2);




```
while (aux.prox != null ) {
    if (aux.prox.item == elemento)
        return aux.prox.item;
    aux = aux.prox;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- A classe Lista contém:
  - uma referência para a célula cabeça;
  - uma referência para a última célula da lista; e
  - uma referência para armazenar a posição corrente na lista.

Implementação de Listas por meio de Estruturas Auto Referenciadas

```
public class ListaRef
  {
    private class CelulaRef
    {
      public Celula item;
      public CelulaRef prox;
    }
    private CelulaRef primeiro, ultimo, pos;
    // Operações

    public ListaRef()
    { // Cria uma Lista vazia
      this.primeiro = new CelulaRef();
      this.pos = this.primeiro;
      this.ultimo = this.primeiro;
      this.primeiro.prox = null;

    }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

```
public bool equals(Celula chave, Celula obj)
    {
        if (chave.get_peso() != obj.get_peso())   return false;

        if (chave.get_vertice() != obj.get_vertice())    return false;

        return true;
    }

    public Celula pesquisa(Celula chave)
    {
        if (this.vazia() || chave == null)
            return null;

        CelulaRef aux = this.primeiro;
        while (aux.prox != null)
        {
            if (this.equals(aux.prox.item, chave))    return aux.prox.item;

            aux = aux.prox;
        }
        return null;
    }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

```
public void insere(Celula x)     {
        this.ultimo.prox = new CelulaRef();
        this.ultimo = this.ultimo.prox;
        this.ultimo.item = x; this.ultimo.prox = null;
    }

    public Celula retira(Celula chave)     {
        if (this.vazia() || (chave == null))
            throw new Exception("Erro : Lista vazia ou chave invalida");

        CelulaRef aux = this.primeiro;
        while (aux.prox != null && !this.equals(aux.prox.item, chave))
            aux = aux.prox;

        if (aux.prox == null)     return null; // não encontrada

        CelulaRef q = aux.prox;
        Celula item = q.item; aux.prox = q.prox;

        if (aux.prox == null)    this.ultimo = aux;

        return item;
    }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

```
public Celula Primeiro()
    {
        this.pos = primeiro;
        return proximo();
    }

    public Celula proximo()
    {
        this.pos = this.pos.prox;
        if (this.pos == null)
            return null;
        else
            return this.pos.item;
    }
    public bool vazia()
    {
        return (this.primeiro == this.ultimo);
    }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

```
public void imprime()
    {
        CelulaRef aux = this.primeiro.prox;
        while (aux != null)
        {
            Console.Write(aux.item.ToString() + " ");
            aux = aux.prox;
        }
    }

    public int quantidade()     {
        CelulaRef aux = this.primeiro.prox;
        int qtd = 0;
        while (aux != null)
        {
            qtd++;
            aux = aux.prox;
        }

        return qtd;
    }

  }
```

Bibliografia

[Cormen et al., 2009] Introduction to Algorithms, 3rd edition. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. 2009, 1312 pp. MIT Press, ISBN 9780262033848.

Cormen et al., 2002] Algoritmos: Teoria e Prática – Tradução da segunda edição em inglês. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. 2002, 936 pp. Editora Campus, ISBN 8535209263.

BOAVENTURA NETTO, Paulo Oswaldo; JURKIEWICZ, Samuel.
Grafos: introdução e prática. Editora Blucher. 2009. ISBN: 9788521204732.

[Sedgewick, 1992] Algorithms in C++. Robert Segdewick. c1992, 656 pp. Boston: Addison Wesley, ISBN 0201510596

Marco Goldbarg; Elizabeth Goldbarg. Grafos - conceitos , algoritmos e aplicações . Campus. 2012.

[Ziviani, 2006] Projeto de Algoritmos com Implementações em Java e C++.
Nivio Ziviani. 2006, 642 pp. Editora Thomson, ISBN 8522105251.

Robin J. Wilson. Introduction to graph theory. Longman, 1996